

Software Test Plan

AWEB Toolchain (`atangle` 2.0 / `aweave` 2.0)

AWEB Toolchain Maintainers

2026-04-30

Document identifier	AWEB-STP-2.0
Status	Approved for Test
Conforming standard	IEEE Std 829-2008

Contents

1 Introduction

1.1 Document identifier

This is the Software Test Plan (STP) for release 2.0 of the AWEB toolchain, comprising the programs `atangle` and `aweave`. It is identified as AWEB-STP-2.0.

1.2 Scope

Release 2.0 extends the toolchain so that AWEB source files may contain Ada 95 (ISO/IEC 8652:1995) programs in addition to the previously supported Ada 83 (ANSI/MIL-STD-1815A-1983) programs. Six reserved words have been added to the lexical recognizer of `aweave`:

`abstract`, `aliased`, `protected`, `requeue`, `tagged`, `until`.

The lexical recognizer of `atangle` did not require change: Ada 95 retains the identifier syntax of Ada 83. This STP exercises both the unchanged behaviour (regression) and the new behaviour (Ada 95 keyword classification), and verifies the surrounding container artefacts (`Dockerfile`, `awebpg.sh`, `docker-entrypoint.sh`).

1.3 References

- R1. IEEE Std 829-2008, *Standard for Software and System Test Documentation*.
- R2. ISO/IEC 8652:1995, *Information technology — Programming languages — Ada* (Ada 95).
- R3. ANSI/MIL-STD-1815A-1983, *Reference Manual for the Ada Programming Language* (Ada 83).
- R4. D. E. Knuth, *Literate Programming*, CSLI, 1992.
- R5. `requirements.txt` in this repository: functional requirements for release 2.0.

R6. `aweave/aweave.aweb`, `atangle/atangle.aweb`: literate sources of the SUT.

2 Test items

ID	Item	Version	Source of record
TI-1	<code>atangle</code> executable	2.0	<code>atangle/atangle.adb</code> (+ tangled units)
TI-2	<code>aweave</code> executable	2.0	<code>aweave/aweave.adb</code> (+ tangled units)
TI-3	<code>Dockerfile</code>	2.0	<code>./Dockerfile</code>
TI-4	<code>awebpg.sh</code> driver	2.0	<code>./awebpg.sh</code>
TI-5	<code>docker-entrypoint.sh</code>	2.0	<code>./docker-entrypoint.sh</code>

Each item is delivered as part of a single OCI image tagged `evansjr/awebpg:2.0`.

3 Features to be tested

ID	Feature
F-1	Build of <code>atangle</code> and <code>aweave</code> from delivered Ada source (<code>gnatmake</code>).
F-2	Banner / version reporting.
F-3	Tangling of an Ada 83 program (regression).
F-4	Tangling of an Ada 95 program containing the six new reserved words.
F-5	Weaving of an Ada 83 program (regression).
F-6	Weaving of an Ada 95 program: each new reserved word is set in <code>\&{...}</code> (bold) face.
F-7	Round-trip: tangling is idempotent on a fixed Ada 95 source.
F-8	Container build (<code>docker build</code>) yields image <code>evansjr/awebpg:2.0</code> .
F-9	PostgreSQL initialization on first container start; persistence across restarts.
F-10	<code>make</code> , <code>file</code> , <code>emacs</code> are present in the container image.
F-11	<code>awebpg.sh</code> dispatch (<code>atangle</code> , <code>aweave</code> , <code>shell</code> , <code>test</code>) functions inside the container.
F-12	The full test harness can be executed inside the container (requirement 7).

4 Features not to be tested

ID	Feature	Rationale
NF-1	Ada 2005/Ada 2012 keywords (<code>interface</code> , <code>overriding</code> , <code>synchronized</code> , <code>some</code>)	Out of scope for release 2.0; only Ada 95 was contracted.
NF-2	Performance / throughput on multi-megabyte AWEB inputs	No performance requirement was levied.
NF-3	Internationalization of TeX output	English/ASCII output only is contracted.
NF-4	TeX rendering pipeline (<code>tex</code> , <code>pdflatex</code>)	Downstream of the SUT; the STP verifies textual output, not typesetting.

5 Approach

The strategy is *black-box functional testing* driven by a shell harness (`tests/test_aweb.sh`) that

1. compiles both programs with `gnatmake` (or detects an existing in-container installation under `/usr/local/bin`),

2. runs each program against curated AWEB fixtures and inspects the output for keyword classification and structural properties,
3. performs a round-trip equivalence test for the new keywords,
4. inspects the container image for tooling and version labels.

Each test case is binary pass/fail. The harness exits with status 0 only if every case passes; it prints a one-line summary per case. It is suitable for CI pipelines and for ad-hoc developer verification. No interactive judgement is required.

5.1 Test levels covered

Unit (lexical-table) F-2, F-4, F-6.

Integration (program + I/O) F-1, F-3, F-5, F-7, F-11.

System (container) F-8, F-9, F-10, F-12.

5.2 Test types

Functional, regression, configuration.

5.3 Execution environments

The harness is designed to run in two execution environments:

1. *Host* — requires Docker for the container-level cases; requires GNAT for the native-compile cases (when not running in the container).
2. *In container* — invoked via `awebpg.sh test` or `awebpg.sh shell` followed by `bash tests/test_aware.sh`. In this mode the harness uses the pre-installed binaries at `/usr/local/bin/atangle` and `/usr/local/bin/aware` and skips the docker-of-docker cases (TC-010 ... TC-014).

6 Item pass/fail criteria

A test item passes when *every* test case mapped to it (see §??) passes. A test case passes when its observed result matches its expected result exactly: `diff -u` empty for textual cases, exit status zero for build cases, presence of artefact for configuration cases.

7 Suspension criteria and resumption requirements

Testing is suspended if

- `gnatmake` fails on unmodified fixtures (toolchain rather than SUT failure),
- the Docker daemon is unreachable when running container-level cases on the host, or
- a fixture file in `tests/` is reported missing.

Testing resumes once the blocking environmental fault is corrected; all previously executed cases shall be re-executed.

8 Test deliverables

Deliverable	Location
This Software Test Plan	<code>SOFTWARE_TEST_PLAN.tex</code>
Test harness	<code>tests/test_aweb.sh</code>
Test fixtures (Ada 83 + Ada 95)	<code>tests/fixtures/</code>
Test log (produced by harness)	<code>tests/test_aweb.log</code>

9 Testing tasks

#	Task	Predecessors
1	Stage source tree	—
2	Build container image	1
3	Run <code>awebpg.sh test</code> (or run the harness on the host)	1, 2
4	Inspect log; declare pass/fail	3

10 Test cases

The fourteen cases below are implemented by `tests/test_aweb.sh`. “Procedure” is summarised; the full procedure is the script itself.

ID	Title	Maps to	Procedure (abbreviated)	Expected result
TC-001	Build atangle	F-1	<code>cd atangle && gnatmake atangle.adb</code>	Exit 0; <code>atangle</code> executable produced.
TC-002	Build aweave	F-1	<code>cd aweave && gnatmake aweave.adb</code>	Exit 0; <code>aweave</code> executable produced.
TC-003	atangle banner	F-2	<code>atangle ada83.aweb ; first stdout line.</code>	Contains “This is ATANGLE, Version 2.0 (Ada 95)”.
TC-004	aweave banner	F-2	<code>aweave ada83.aweb ; first stdout line.</code>	Contains “This is AWEAVE, Version 2.0 (Ada 95 keyword support)”.
TC-005	Tangle Ada 83 fixture	F-3	Tangle <code>ada83.aweb</code> ; check produced <code>.a</code> file.	Output produced; no errors.
TC-006	Tangle Ada 95 fixture	F-4	Tangle <code>ada95.aweb</code> ; check produced <code>.a</code> file.	Output produced; no errors.
TC-007	Weave Ada 83 fixture (regression)	F-5	Weave <code>ada83.aweb</code> ; check that <code>\&{begin}</code> , <code>\&{end}</code> appear in the <code>.tex</code> .	Both control sequences present.
TC-008	Weave Ada 95: new keywords are bold	F-6	Weave <code>ada95.aweb</code> ; verify <code>\&{abstract}</code> , <code>\&{aliased}</code> , <code>\&{protected}</code> , <code>\&{requeue}</code> , <code>\&{tagged}</code> , <code>\&{until}</code> each appear at least once in the <code>.tex</code> .	All six present.
TC-009	Round-trip equivalence	F-7	Tangle <code>ada95.aweb</code> twice; <code>diff</code> the two outputs.	<code>diff</code> empty.

ID	Title	Maps to	Procedure (abbreviated)	Expected result
TC-010	Container build	F-8	<code>docker build -t evansjr/awebpg:2.0 .</code>	Build succeeds; image carries label <code>org.opencontainers.image.version=2.</code>
TC-011	Container has make, file, emacs	F-10	<code>docker run ... command -v make file emacs.</code>	All three resolve to absolute paths.
TC-012	PostgreSQL initialisation & persistence	F-9	First <code>docker run</code> initialises <code>\$PGDATA</code> ; second run on the same volume reuses it.	First run logs <i>Initializing</i> ; second run does not.
TC-013	<code>awebpg.sh</code> dispatch	F-11	<code>awebpg.sh atangle ada95.aweb</code> inside the container.	Exit 0; <code>web_output.a</code> produced.
TC-014	In-container Ada 95 keyword classification	F-6, F-12	Run <code>aweave ada95.aweb</code> in the container; check the produced <code>.tex</code> contains <code>\&{abstract}</code> , <code>\&{aliased}</code> , <code>\&{protected}</code> , <code>\&{requeue}</code> , <code>\&{tagged}</code> , <code>\&{until}</code> .	All six classified as reserved (independent of host GNAT).

11 Environmental needs

- Host with Docker ≥ 24 and ≥ 2 GiB RAM.
- GNAT (any FSF release with GCC ≥ 9) for non-container runs of TC-001...TC-009. Inside the container GNAT is provided by the image and the user need not install anything on the host.
- Coreutils (`diff`, `grep`, `sed`, `cmp`) for the harness.

12 Responsibilities

Role	Party
Test author	Toolchain maintainer
Test executor	CI system or developer running <code>awebpg.sh test</code>
Defect adjudicator	Toolchain maintainer

13 Staffing and training

No specialised training is required beyond the ability to read AWEB source, Ada source, and shell scripts.

14 Schedule

The harness is self-contained and intended to run on every commit; a full pass typically completes in under thirty seconds on a developer workstation, plus the time to build the container image (≈ 2 minutes on first build, seconds when cached).

15 Risks and contingencies

Risk	Mitigation
Future GNAT release reserves additional Ada 2022 keywords used as identifiers in legacy AWEB source.	Restrict regression fixtures to identifiers known not to clash; pin the GNAT version in the Dockerfile .
Diff-based weaving comparison too brittle (whitespace).	Compare on a control-sequence whitelist instead of full text.
Docker daemon unavailable in CI environment.	Container-level cases (TC-010...TC-014) skip with explicit SKIP status; the harness still asserts non-container cases pass.

16 Approvals

Role	Name	Date
Author	(toolchain team)	2026-04-30
Configuration mgr.	(toolchain team)	2026-04-30
Quality assurance	(toolchain team)	2026-04-30